LLM LOG

Catch the Rocket – FCA Mid-Semester Hackathon

Team: ComputeX

Team Members:

1. Sourabha B N – 24UG00381
2. C Pranusha – 24UG00445
3. Nayeem Ahmed Didagur – 24UG00555

Course: Fundamentals of Computer Architecture

Date: 2 October 2026

1. Purpose of Using the LLM

The LLM was used as a supporting tool during the development of the Catch the Rocket ground station. It was mainly used to:

- Understand the hackathon requirements.
- Analyse the official rocket tracker.
- Verify mathematical calculations.
- Understand the least-squares approach for the descent line.
- Understand quadratic fitting for the rocket's coasting trajectory.
- Check expected values for gravity and apogee.
- Discuss ESP32 memory constraints.
- Understand CPU-cycle measurement.
- Identify possible implementation issues.
- Prepare explanations for the presentation and questioning.

The LLM was used for understanding and debugging rather than blindly copying an implementation. The suggestions were tested against the official tracker and the ESP32 results.

2. LLM Interaction Log

Interaction 1 – Understanding the Problem

Prompt

Analyze the Catch the Rocket hackathon PDF and explain what the project requires, including the memory limit, number of readings, commands, accuracy requirements, and performance requirements.

LLM Suggestion

The LLM explained that the ESP32 must process exactly **100,000 altitude readings**, cannot store all the readings, and must keep the required state inside a single structure of at most **4096 bytes**.

It also identified the main commands:

```
G
APOGEE
DEPLOY
DESCENT
LAND
ALT t
GLITCHES
STATS
```

The LLM also highlighted the **12,000 CPU-cycle-per-reading** requirement.

Result

Correct.

The explanation matched the project specification.

3. Interaction 2 – Calculating the Official Tracker Answers

Prompt

Calculate the expected G, APOGEE, DEPLOY, DESCENT and LAND outputs from the official tracker constants.

LLM Suggestion

The LLM calculated the official tracker values as approximately:

```
G        = 9.810 m/s²
APOGEE   = 8.002 s, 626.08 m
DEPLOY   = 12.000 s, 547.68 m
DESCENT  = -4.50 m/s
LAND     = 133.707 s
```

Result

Correct for checking the official tracker.

These values were used as reference values for validating the implementation.

The PDF itself provides the official tracker constants and states that they may be used to check the answers, while the submitted program must not hard-code them.

4. Interaction 3 – Understanding DESCENT

Prompt

How can we calculate the parachute descent velocity without storing all 100,000 readings?

LLM Suggestion

The LLM explained that the readings after 45 seconds are guaranteed to be under the parachute. Therefore, those readings can be treated as a straight-line relationship:

$h=a+bth=a+bt$

where:

- aa is the intercept.
- bb is the descent velocity.

The required least-squares quantities can be accumulated while the readings arrive instead of storing every altitude.

Result

Correct.

This approach agrees with the Level 1 requirement that readings from $t=45t=45$ seconds onward can be used for a straight-line descent fit.

5. Interaction 4 – Understanding LAND

Prompt

Once the descent line is known, how can the predicted landing time be calculated?

LLM Suggestion

The LLM explained that the descent can be represented as:

$h=a+bth=a+bt$

For landing:

$h=0h=0$

Therefore:

$$t_{land} = -\frac{a}{b}$$

The result should be checked against the expected physical flight behaviour.

Result

Correct.

The resulting implementation produced:

```
LAND = 133.7089 s
```

which was consistent with the expected official-tracker value.

6. Interaction 5 – Understanding Gravity and APOGEE

Prompt

How can gravity and apogee be calculated using the first 8 seconds of rocket data?

LLM Suggestion

The LLM explained that during the initial coasting period the trajectory follows a quadratic:

$$h = a + bt + ct^2$$

The coefficients can be obtained using least-squares fitting.

The relationship with the physical equation is:

$$h = h_0 + v_0 t - \frac{1}{2} g t^2$$

Therefore:

$g=-2cg=-2c$

The velocity is:

$v(t)=b+2ctv(t)=b+2ct$

At apogee:

$v(t)=0v(t)=0$

so:

$tapogee=-b2ct_{apogee}=-\frac{b}{2c}$

The corresponding height can then be obtained by substituting the apogee time into the fitted quadratic.

Result

Correct approach.

The implementation produced:

```
G = 9.8138 m/s²
APOGEE time = 8.0006 s
APOGEE height = 626.0644 m
```

These results were within the required Level 2 accuracy limits.

The project specification explicitly recommends fitting a quadratic to the first 8 seconds for Level 2.

7. Interaction 6 – Memory Optimization

Prompt

How can we perform least-squares calculations when we cannot store the 100,000 readings and have only 4 KB of memory?

LLM Suggestion

The LLM suggested using **running sums** rather than storing individual readings.

For a straight-line fit, quantities such as:

Σt
Σh
Σt^2
Σth

can be accumulated while readings arrive.

For the quadratic fit, additional sums such as:

Σt^3
Σt^4
$\Sigma t^2 h$

are required.

The final coefficients can then be calculated from these accumulated quantities.

Result

Useful and correct.

This approach allowed the implementation to use a compact state rather than storing the raw readings.

8. Interaction 7 – ESP32 Performance

Prompt

Explain how the ESP32 cycle limit should be measured for each reading.

LLM Suggestion

The LLM explained that the cycle counter should be read before and after the processing associated with each reading, using:

```
ESP.getCycleCount()
```

The difference should be treated as an unsigned 32-bit value.

The important value is the **worst-case reading**, not simply the average.

Result

Correct.

The implementation measured:

```
Worst-case cycles = 9016
Required maximum  = 12000
```

Therefore:

```
9016 < 12000
```

and the current implementation was within the specified processing budget.

The specification specifically requires measuring the slowest reading and states that every reading must finish within the 12,000-cycle budget.

9. Interaction 8 – Checking the Current STATS

Prompt

Interpret these STATS results: 100000 readings, 55000 Level 1 readings, 8000 Level 2 readings, state size 152 bytes and worst cycles 9016.

LLM Suggestion

The LLM interpreted the results as:

```
Total readings = 100000
State size = 152 bytes
Maximum state = 4096 bytes
Worst-case cycles = 9016
Maximum cycles = 12000
```

The remaining cycle margin is:

$$12000-9016=298412000\text{-}9016=2984$$

cycles.

Result

Correct interpretation.

The current implementation is within both the memory and measured performance limits.

10. Interaction 9 – Level 3 Planning

Prompt

How should we approach Level 3 and determine the parachute deployment point?

LLM Suggestion

The LLM suggested separating the flight into two models:

```
Coasting phase     → quadratic model
Parachute phase    → straight-line model
```

The deployment point can then be estimated from the transition between these two behaviours.

The LLM also suggested validating the result against the tracker guarantees instead of assuming that the official deployment time will always be 12 seconds.

Result

Useful, but requires testing.

The idea matches the Level 3 requirement to identify where the coast ends and determine the crossing between the two fitted curves.

11. Interaction 10 – Glitch Handling

Prompt

What should be considered when adding glitch detection in Level 4?

LLM Suggestion

The LLM pointed out that normal readings have noise within approximately ±2 m, while glitches are at least 50 m away from the true altitude.

Therefore, the implementation needs to distinguish ordinary measurement noise from large abnormal deviations without incorrectly treating the transition to parachute descent as a glitch.

Result

Partially useful; requires experimental verification.

The LLM suggestion provided the general direction, but the final detection method must be tested against the official and practice trackers because the specification requires exact glitch counting and correct answers with glitches enabled.

The tracker guarantees specify at most 1% glitches, a minimum separation of 10 readings between glitches, and no glitches during the first 1,000 readings.

12. One Important Limitation / Incorrect Assumption

During development, the LLM initially treated the official tracker values as if they could directly be used as implementation constants.

This was corrected after checking the project rules.

The specification clearly states that the official tracker constants can be used to **check answers**, but the program must not use those constants because the judges will use different tracker values.

Lesson Learned

The solution must derive all answers from the incoming readings and must work with an unknown tracker.

13. What the Team Learned from Using the LLM

The LLM was particularly useful for:

1. Understanding the mathematical model of the rocket.
2. Converting the physical equations into least-squares fitting problems.
3. Understanding how running sums can replace storage of raw readings.
4. Checking mathematical calculations.
5. Understanding ESP32 cycle measurement.
6. Interpreting memory and performance statistics.
7. Planning the transition from Level 2 to Level 3.
8. Identifying possible issues with glitch detection.

However, all important suggestions were checked against the official tracker and actual ESP32 results.

14. Final LLM Log Summary

Interaction	Topic	Result
1	Project requirements	Correct
2	Official tracker calculations	Correct
3	DESCENT calculation	Correct
4	LAND calculation	Correct
5	G and APOGEE	Correct
6	Memory optimization	Useful and correct
7	CPU-cycle measurement	Correct
8	STATS interpretation	Correct
9	Level 3 deployment approach	Useful, requires testing
10	Glitch handling	Partially useful, requires testing

11	Hard-coded tracker values	Corrected
12	Overall project understanding	Improved

Final Observation

The LLM helped the team understand the mathematics, memory constraints, performance requirements, and debugging process. The team did not rely solely on the LLM's answers; the important calculations and implementation behaviour were verified using the ESP32 and the official tracker.

This follows the project requirement to keep a log of important LLM prompts, suggestions, and whether they were correct.